안녕하세요.
Jquery 강좌중인 Falcon 입니다.

이번강좌는 여러분들이 앞으로 이벤트를 활용하기 위해서는 알고 가셔야하는 부분 입니다.

```
<body>
  <fieldset>
    <legend>event propagation</legend>
    <input type="button" id="target" value="target">
  </fieldset>
</body>
```

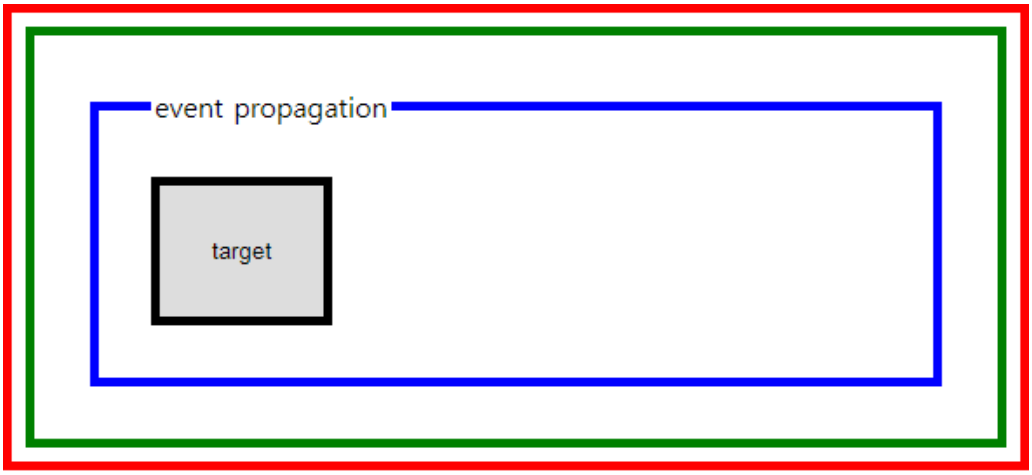
index.html

위에서 여러분들이 중요하게 봐야할 내용은 HTML의 부모와 자식 관계 입니다. 위에서는 body가 부모이지만 이 body의 부모는 html이라는 사실도 이미 여러분들은 알고 계실 겁니다. 그런데 여러분들이 이벤트라는 것을 제대로 활용하기 위해서는 한가지 더 알고 계셔야하는 것이 있습니다. 부모와 자식이 동시에 이벤트를 가지고 있다면 그 이벤트는 부모와 자식 관계를 통해서 자동적으로 이벤트가 발생하게 된다는 것 입니다.

```
<style>
    html{border:5px solid red;padding:30px;}
    body{border:5px solid green;padding:30px;}
    fieldset{border:5px solid blue;padding:30px;}
    input{border:5px solid black;padding:30px;}
</style>
```

style.css

위를 보시면 html 자식으로 body가 있고, body의 자식으로 fieldset이라는 이벤트가 있고, fieldset이라는 자식으로는 input이 있습니다. 그를 구분 짓게 css를 통해서 이렇게 그려보았습니다.



개인적으로 이유는 모르겠지만 <html>태그에 적용한 css가 이 코드를 실행했을 때 저는 적용되지 않았습니다. 위에서 보여드리는 것은 개발자 도구를 통해서 직접적으로 적용시켰음을 알려드립니다. 위에서 부모와 자식 관계를 이렇게 테두리를 통해서 여러분들에게 보여드리고 있습니다. 당연히 이벤트를 발생 시켜야하기 때문에

```
1   window.onload = function() {
2     function handler(event){
3         var phases = ['capturing', 'target', 'bubbling'];
4         console.log(event.target.nodeName, this.nodeName, phases[event.eventPhase-1]);
5     }
6     document.getElementById('target').addEventListener('click', handler, true);
7     document.querySelector('fieldset').addEventListener('click', handler, true);
8     document.querySelector('body').addEventListener('click', handler, true);
9     document.querySelector('html').addEventListener('click', handler, true);
10  };
```

master.js

javascript도 불러오도록 하겠습니다. 위를 보면 addEventListenter() 메소드에 3번째 인자값이 true 입니다. 이는 이벤트의 발생 조건을 위에서 아래로 실행하는 캡처링의 방식으로 적용하겠다는 의미 입니다. 지금 이 문구를 이해하지 못해도 상관 없습니다. 6행 ~ 9행이 실행하는 것은 handler이라는 함수를 호출하죠.

```
INPUT HTML capturing
INPUT BODY capturing
INPUT FIELDSET capturing
INPUT INPUT target
```

위의 결과에서 중요한 것은 저는 단지 input태그의 target이라는 버튼을 한번 눌렀습니다. 그런데 위의 결과는 html → body → fieldset → input 순으로 전부 호출되는 것을 볼 수 있습니다. 즉, 부모에서 부터 자식까지 전달되는 동안에 존재하는 이벤트가 실행된다는 것을 알 수 있습니다. 이렇게 부모에서부터 자식으로 전달되는 이벤트 방식을 캡처링이라고 합니다. 그러나 이 캡처링은 기본 값이 아니기도 하지만 실제로 많이 사용되지 않는 방식이라고 합니다. 그렇다면 기본 값이면서 앞으로 여러분들을 매우 괴롭게 할 버블링이라는 것에 대해서도 한번 실행을 해봅시다. 코드에 대한 설명은 아래에서 해드리겠습니다.

```
1   window.onload = function() {
2     function handler(event){
3         var phases = ['capturing', 'target', 'bubbling'];
4         console.log(event.target.nodeName, this.nodeName, phases[event.eventPhase-1]);
5     }
6     document.getElementById('target').addEventListener('click', handler, false);
7     document.querySelector('fieldset').addEventListener('click', handler, false);
8     document.querySelector('body').addEventListener('click', handler, false);
9     document.querySelector('html').addEventListener('click', handler, false);
10  };
```

master.js

위와달라지는 것은 3번째 인자 값이 true에서 false로 변한 것 외에는 코드가 달라진 것은 하나도 없습니다. 그런데 이 코드를 실행하게 되면 결과가 달라지게 됩니다. 대체 무엇이 달라지는지 한번 input태그를 실행해보도록 하죠.

INPUT INPUT target

INPUT FIELDSET bubbling

INPUT BODY bubbling

INPUT HTML bubbling

결과를 보면 아까와는 달리 부모에서 자식으로 전달되는 방식이 아닌 자식에서부터 부모로 올라가는 방식으로 전달 됩니다. input → fieldset → body → html 순으로 출력되는 것을 위에서 보이고 있습니다. 그리고 이게 기본 값입니다. 여기서 여러분들이 조금 궁금해 하실 부분들이 있는데 바로 handler 메소드이실 겁니다.

```
2   function handler(event){
3       var phases = ['capturing', 'target', 'bubbling'];
4       console.log(event.target.nodeName, this.nodeName, phases[event.eventPhase-1]);
5   }
```

master.js 일부

이 부분에 대해서 잠깐 설명해드리자면 event.target 과 this에 관해서 입니다. 위의 결과 값을 보면 event.target.nodeName은 INPUT이 출력이 되는데 this.nodeName은 변하고 있습니다. 그 이유는 event.target은 제가 위에서 누든 위치 입니다. 조금 쉽게 이해를 할 수 있는 방법은 여러분들이 이코드를 작성해서 실행중이시라면 파란색 부분에서 클릭을 한번 해보시길 바랍니다. 그러면 이러한 결과값이 뜨게 됩니다.

FIELDSET FIELDSET target

FIELDSET BODY bubbling

FIELDSET HTML bubbling

즉, event.target은 여러분들이 클릭한 그 위치라는 것을 알 수 있습니다. this는 자기 자신을 가리키게 됩니다. 쉽게 body에서 발생한 이벤트는 자기 자신 body가 this 입니다. 이 부분은 사실 여러분들이 많은 코딩을 하다보면 이해가 되실 겁니다. 마지막으로 3행에 보면 phases라는 배열을 하나 만들어 두었습니다. 그리고 출력되는 부분을 보면 event.eventPhase가 보이는데 이는 여러분들이 호출한 값이 캡처링이라면 1을, 자기자신이라면 2를, 버블링이라면 3을 반환하게 됩니다. 거기에 -1을 하면 배열의 값과 동일해지면서 출력해주는 형태 입니다.

```
1   window.onload = function() {
2     function handler(event){
3         var phases = ['capturing', 'target', 'bubbling'];
4         console.log(event.target.nodeName, this.nodeName, phases[event.eventPhase-1]);
5     }
6     function stophandler(event){
7       var phases = ['capturing', 'target', 'bubbling'];
8       console.log(event.target.nodeName, this.nodeName, phases[event.eventPhase-1]);
9       event.stopPropagation();
10  }
11    document.getElementById('target').addEventListener('click', handler, false);
12    document.querySelector('fieldset').addEventListener('click', handler, false);
13    document.querySelector('body').addEventListener('click', stophandler, false);
14    document.querySelector('html').addEventListener('click', handler, false);
15  };
```

master.js

위에서 끝내려고 했으나 한가지 더 알려드릴 부분이 있어서 이렇게 글을 수정하게 되었습니다. 위에서 보면 stophandler이라는 메소드를 추가했습니다. 9행 부분을 보면 stopPropagation이라는 메소드라는 것이 보이는데 이는 더이상 내 위의 부모에게 이벤트를 전파하지 않겠다라는 의미 입니다.

```
INPUT INPUT target
INPUT FIELDSET bubbling
INPUT BODY bubbling
```

13행에 body에는 stophandler이라는 함수를 호출하는데 여기에 stopPropagation()이라는 메소드가 있기 때문에 이 위에 존재하는 HTML에 대한 이벤트가 나오지 않는 것을 볼 수 있습니다. 이 부분도 알고 넘어가시면 좋을 거 같아 급하게 글을 수정해서 보여드립니다. 그럼 저는 이만~!

이것으로 [Jquery 강좌] 제 31강 버블링과 캡처링 편을
마치도록 하겠습니다.